RRT* Motion Planning with Control Barrier Function Safety Derived from Poisson Fields

Maciej Różewicz

***Abstract*— This paper proposes a novel framework for safe motion planning that integrates Control Barrier Function (CBF) safety constraints with the Rapidly-Exploring Random Tree Star (RRT*) algorithm. The CBF is derived directly from the solution of a Poisson equation defined over the free workspace, referred to as a *Poisson Safety Function (PSF)*. The PSF encodes obstacle boundaries and safety gradients in a smooth, differentiable field that naturally satisfies Laplace-type spatial regularity. The proposed method, termed *Safe-CBF-RRT**, uses the PSF and its spatial derivatives to enforce CBF-based safety conditions along sampled edges during tree expansion. Simulation results demonstrate that the method enhances safety and convergence properties compared to standard RRT*, while maintaining computational tractability.**

I. Introduction

In modern robotics applications, ensuring safe navigation in complex environments is one of the major challenges. This is particularly critical in scenarios involving human-robot interaction, autonomous vehicles, and unstructured terrains. Traditional motion planning algorithms often focus on finding feasible paths without formal guarantees of safety. To achieve such provable safety, Control Barrier Functions (CBFs) have emerged as a powerful tool for enforcing safety constraints in control systems in last years.

Sampling-based motion planners such as RRT or RRT* are widely used for complex robotic systems due to their scalability and ability to handle nonconvex constraints. However, they rely primarily on geometric collision checking, offering no formal guarantees of safety. For this reason, integrating CBFs into sampling-based planners seems a promising direction to enhance safety while retaining the benefits of sampling-based methods.

A critical challenge in using CBFs for planning is the construction of a suitable safety function $h(x)$, which must be continuously differentiable and satisfy certain regularity conditions. Analytical definitions of $h(x)$ exist for simple geometries, and lately there have been attempts to get CBF as solution of Poisson equation [**?**]. This approach will be used in this paper to generate a Poisson Safety Function (PSF) that encodes obstacle boundaries and provides smooth safety gradients. Such definition of $h(x)$ is particularly advantageous for complex environments where analytical forms are not feasible. More over seems to be naturally compatible with the RRT or RRT* framework.

Contributions: In this the *Safe-CBF-RRT** framework, the following contributions are made:

- Integration of the PSF into a modified RRT* algorithm.
- Empirical results showing improved safety margins and convergence.

Department of Automatic Control and Robotics, AGH University of Science and Technology, Kraków, Poland. Email: mrozewicz@agh.edu.pl

II. Related Work

Sampling-based methods such as RRT and RRT* [**?**] have become standard in high-dimensional motion planning. Extensions incorporating optimality, dynamics, and risk awareness exist but often lack formal safety guarantees.

Potential field and PDE-based navigation [**?**] have long been studied for path planning. Harmonic potential fields, governed by Laplace's equation, ensure smoothness and avoid local minima, yet they do not directly provide a safety margin measurable in the CBF framework.

Control Barrier Functions [**?**], [**?**] ensure set invariance in control systems but require a known differentiable safety function. Existing methods often define $h(x)$ as analytic distances to obstacles, which may be nondifferentiable in complex maps. Our approach bridges this gap by generating $h(x)$ through a Poisson PDE that respects boundary geometry.

[**?**]

III. Preliminaries

A. Control Barrier Functions

Control Barrier Functions (CBFs) provide a formalism for ensuring safety in control systems by enforcing forward invariance of a safe set. Given a dynamical system

$$\dot{x} = f(x) + g(x)u, \tag{1}$$

where $x \in \mathbb{R}^n$ is the state, $u \in \mathbb{R}^m$ is the control input, and f, g are locally Lipschitz continuous functions, a set

$$\mathcal{C} = \{x \in \mathbb{R}^n : h(x) \geq 0\} \tag{2}$$

is defined as *safe* if there exists a continuously differentiable function $h : \mathbb{R}^n \to \mathbb{R}$ such that for all $x \in \mathcal{C}$,

$$\sup_{u \in U} [L_f h(x) + L_g h(x) u] \geq -\kappa h(x), \tag{3}$$

where $L_f h$ and $L_g h$ are the Lie derivatives of h along f and g, respectively, and $\kappa > 0$ is a tuning parameter. This condition ensures that the system's trajectories remain within the safe set $\mathcal{C}$ for all future times.

Algorithm 1 Safe-CBF-RRT*

Initialize tree with x_{start}
for $k = 1 : N_{iter}$ **do**
  Sample x_{rand}
  $x_{near} \leftarrow$ Nearest(x_{rand})
  $x_{new} \leftarrow$ Steer(x_{near}, x_{rand})
  if CollisionFree(x_{near}, x_{new}) and CBFSafe(x_{near}, x_{new}) **then**
    AddNode(x_{new}); RewireNeighborhood(x_{new})
  end if
  if x_{new} near x_{goal} **then**
    Connect and return path
  end if
end for

B. Poisson Safety Function Generation

Let $\Omega \subset \mathbb{R}^2$ denote the workspace with boundary $\partial\Omega$ and obstacles $\mathcal{O}_i$. We define a vector potential field $\mathbf{u} = [u_x, u_y]^T$ solving

$$\left\{ \begin{array}{ll} \nabla^2 \mathbf{u} = 0 & \text{in } \Omega \\ \mathbf{u} = c_i(\mathbf{x} - \mathbf{c}_i) & \text{on } \partial\mathcal{O}_i \end{array} \right\}. \tag{4}$$

A scalar field $h(x, y)$, termed the *Poisson Safety Function (PSF)*, is then obtained by solving

$$\left\{ \begin{array}{ll} \nabla^2 h = -\|\nabla \mathbf{u}\| & \text{in } \Omega \\ h|_{\partial\Omega} = 0 & \text{on } \partial\mathcal{O}_i \end{array} \right\}, \tag{5}$$

ensuring $h > 0$ in free space. Gradients $\nabla h = [\partial h/\partial x, \partial h/\partial y]$ are computed using finite differences.

C. Control Barrier Function Constraint

For a path segment between points a and b, sampled along $p(s) = a + s(b - a)$, $s \in [0, 1]$, the discrete CBF condition is

$$\nabla h(p) \cdot (b - a) \geq -\kappa h(p)\|b - a\|, \tag{6}$$

where $\kappa > 0$ controls conservatism. This ensures that motion along the edge does not approach unsafe regions too rapidly.

IV. Safe CBF-RRT* Algorithm

Safe CBF-RRT* extends RRT* by filtering edges through CBF validation and weighting path costs by safety:

$$J(a, b) = cL + (1 - c)\frac{1}{\bar{h}}, \tag{7}$$

where $L = \|b-a\|$, $\bar{h}$ is mean h along the edge, and $c \in [0, 1]$ balances length and safety.

The planner iteratively samples, extends, and rewires as in RRT*, rejecting edges that violate the CBF condition. A pseudocode outline is shown in Algorithm **??**.

V. Results

A. Simulation Setup

Experiments were conducted in MATLAB using the PDE Toolbox. The workspace was a unit square with three circular obstacles. The grid resolution was 100×100. Parameters used were stepSize $= 1$, $\kappa = 20$, $c = 0.1$, and $N_{iter} = 5000$.

B. Performance

Two configurations were tested:

1) RRT* without CBF safety (baseline)

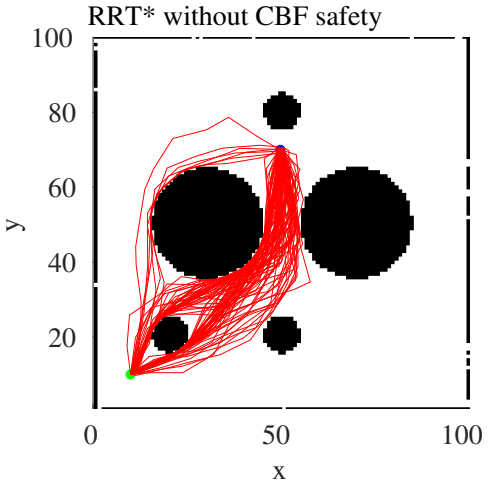

Fig. 1. Exemplary 100 different runs of standard RRT* algorithm. It can be observed that some paths pass very close to obstacles.

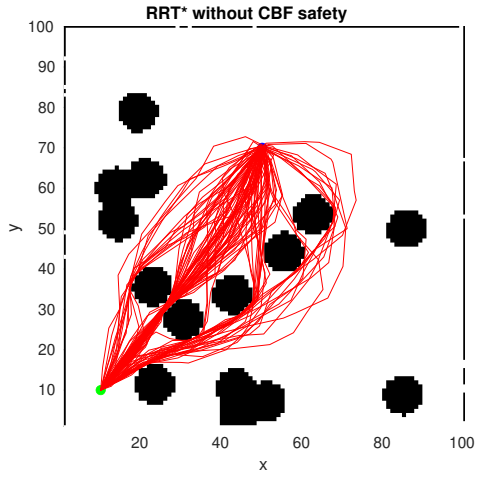

Fig. 2. Exemplary 100 different runs of standard RRT* algorithm. It can be observed that some paths pass very close to obstacles.

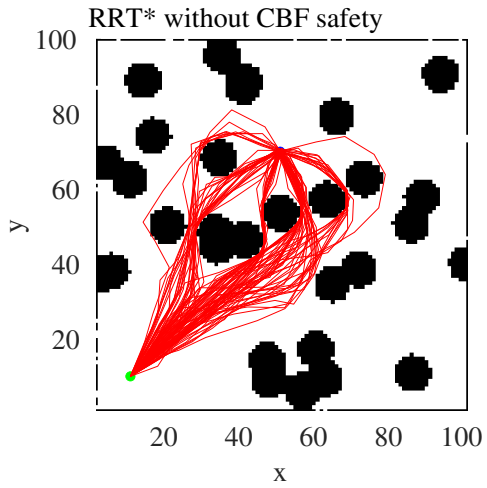

Fig. 3. Exemplary 100 different runs of standard RRT* algorithm. It can be observed that some paths pass very close to obstacles.

2) Safe-CBF-RRT* (proposed)

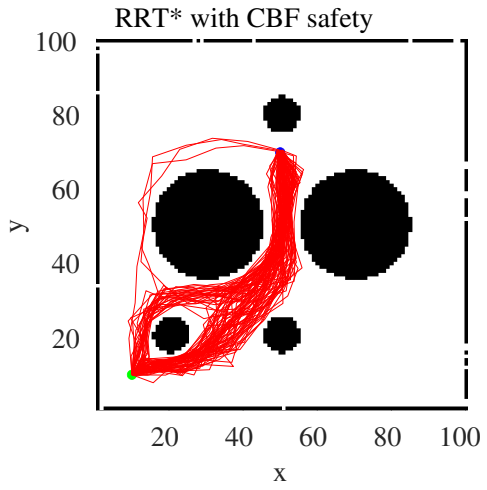

Fig. 4. Exemplary 100 different runs of modified CBF-RRT* algorithm. It can be observed that in this case no path passes close to obstacles.

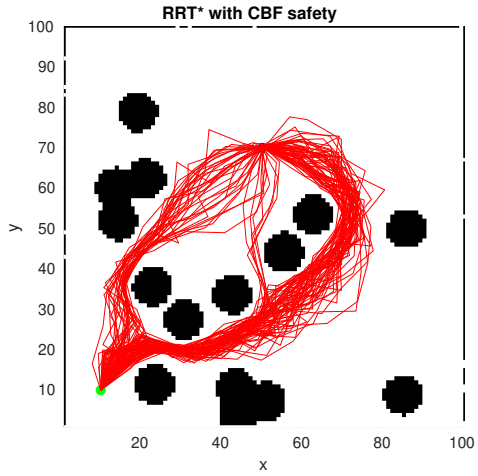

Fig. 5. Exemplary 100 different runs of modified CBF-RRT* algorithm. It can be observed that in this case no path passes close to obstacles.

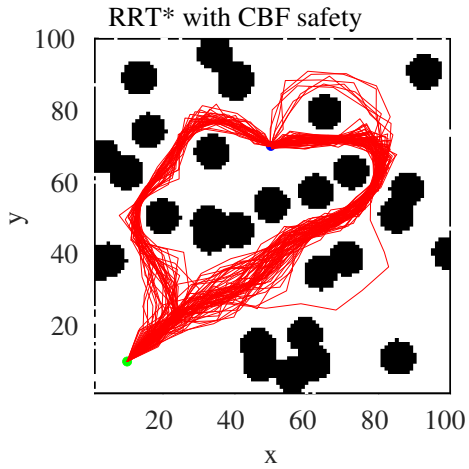

Fig. 6. Exemplary 100 different runs of modified CBF-RRT* algorithm. It can be observed that in this case no path passes close to obstacles.

The PSF produced smooth gradients and continuous safety margins. The proposed planner consistently generated paths that maintained higher safety levels while remaining within 10% of the shortest-path length.

C. Discussion

The CBF constraint effectively rejected paths approaching obstacles too closely. The Poisson field provided well-conditioned gradients, improving planner stability. Despite the additional checks, runtime increased only marginally due to efficient gridded interpolation.

VI. Conclusion and Future Work

We presented *Safe-CBF-RRT**, a motion planning framework combining RRT* with a Control Barrier Function derived from a Poisson Safety Field. The approach unifies PDE-based potential field generation with formal safety guarantees. Future work will extend this method to kino-dynamic models, uncertain environments, and real robotic experiments.